

TOOLS OF THE TRADE

Embracing Our Inner Craftsman

August 23, 2024

Joel FALCOU



A Brief History of Tools

The Original Silicon Valley



A Long Time Ago, In a Valley Far Far Away

Silicium Based Tools

- Flint and later bone based tools are a give-away hints of these periods.
- If they first ensured survival, they also were designed as early social status objects.
- Tools of various level of complexity can be found all over Africa, Europe and Asia.



Two-sided flint ~500'000 BCE



Stone Tools ~12'000 BCE

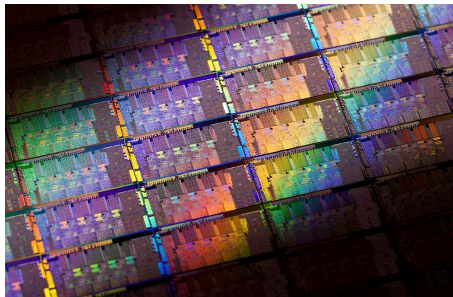
More Recently, In Another Valley Far Far Away

Silicium Based Tools

- Modern stone tools do computation with, obviously, a far better scale factor.
- CPUs are very flat flints that do computations when struck by lightning.
- *Insert joke about Moore's Law having been right for 10^5 years.*



Two-sided flint ~500'000 BCE



Silicium-based calculator ~2000 CE

Why Making Tools?

Tools as know-how encapsulation

- We build tools to avoid repeating complex, error-prone process.
- We build tools so that others can avoid the F^{***} *Around and Find Out* stage.
- Tools are meant to speed-up the pace at which new comers can become proficient at a given task.

Why Making Tools?

Tools as know-how encapsulation

- We build tools to avoid repeating complex, error-prone process.
- We build tools so that others can avoid the *F*** Around and Find Out* stage.
- Tools are meant to speed-up the pace at which new comers can become proficient at a given task.

But what about software tools ?

- They can simplify how we access the power of our *Very Thin Flints*.
- They can help speed-up research by making pre or post-processing easier.
- They can help understand why something is not working as it should.

Why Making Tools?

Tools as know-how encapsulation

- We build tools to avoid repeating complex, error-prone process.
- We build tools so that others can avoid the *F*** Around and Find Out* stage.
- Tools are meant to speed-up the pace at which new comers can become proficient at a given task.

But what about software tools ?

- They can simplify how we access the power of our *Very Thin Flints*.
- They can help speed-up research by making pre or post-processing easier.
- They can help understand why something is not working as it should.

So What Is Talk About ?

- **How our team tried to design HPC tools for over a decade?**
- **What were our good and bad experiences?**

Designing HPC Tools - Attempt #1



Designing Tools for Scientific Computing

Objectives

1. Be non-disruptive.
2. Domain driven optimizations.
3. Provide intuitive API for the user.
4. Support a wide architectural landscape.
5. Be efficient.

Designing Tools for Scientific Computing

Objectives

1. **Be non-disruptive.**
2. Domain driven optimizations.
3. Provide intuitive API for the user.
4. Support a wide architectural landscape.
5. **Be efficient.**

Our Approach

- **Design tools as C++ libraries.**

Designing Tools for Scientific Computing

Objectives

1. Be non-disruptive.
2. **Domain driven optimizations.**
3. **Provide intuitive API for the user.**
4. Support a wide architectural landscape.
5. Be efficient.

Our Approach

- Design tools as C++ libraries.
- **Design these libraries as Domain Specific Embedded Languages (DSEL)**

Designing Tools for Scientific Computing

Objectives

1. Be non-disruptive.
2. Domain driven optimizations.
3. **Provide intuitive API for the user.**
4. **Support a wide architectural landscape.**
5. Be efficient.

Our Approach

- Design tools as C++ libraries.
- Design these libraries as Domain Specific Embedded Languages (DSEL).
- **Use Parallel Programming Abstractions as parallel components.**

Designing Tools for Scientific Computing

Objectives

1. Be non-disruptive.
2. Domain driven optimizations.
3. Provide intuitive API for the user.
4. Support a wide architectural landscape.
5. **Be efficient.**

Our Approach

- Design tools as C++ libraries.
- Design these libraries as Domain Specific Embedded Languages (DSEL).
- Use Parallel Programming Abstractions as parallel components.
- **Use Generative Programming to deliver performance.**

Why Parallel Programming Models?

- **Unstructured parallelism is error-prone.**
- Low level parallel tools are non-composable.
- Provide an abstraction layers over vendor-specific details.
- Provide a framework in which reasoning about parallelism is easier.
- Ex: BSP, Skeletons, Futures and Promises.

Why Parallel Programming Models?

- Unstructured parallelism is error-prone.
- Low level parallel tools are non-composable.
- Provide an abstraction layers over vendor-specific details.
- **Provide a framework in which reasoning about parallelism is easier.**
- Ex: BSP, Skeletons, Futures and Promises.

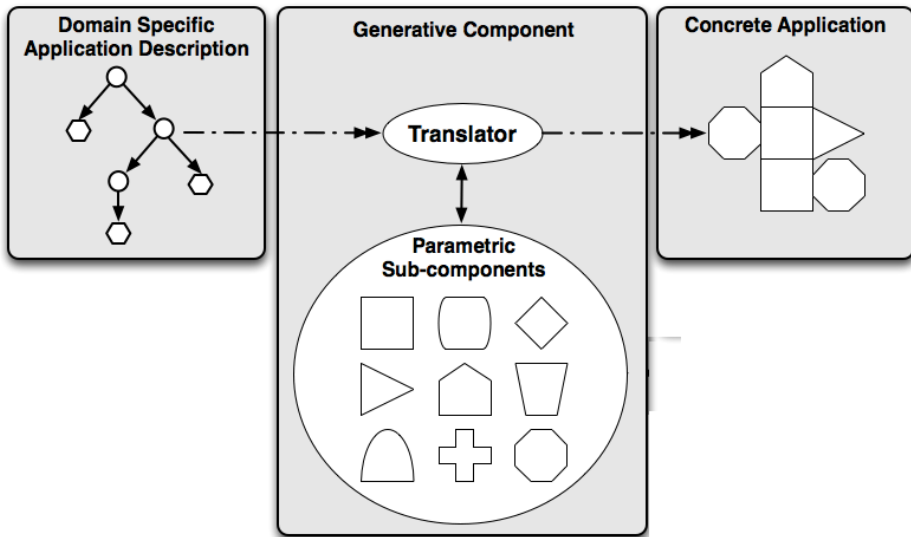
Why Parallel Programming Models?

- Unstructured parallelism is error-prone.
- Low level parallel tools are non-composable.
- Provide an abstraction layers over vendor-specific details.
- Provide a framework in which reasoning about parallelism is easier.
- Ex: BSP, Skeletons, Futures and Promises.

Why DSELs and Other Template-Based Voodoo?

- DSLs incorporate **domain-specific** notations/abstractions into language design [Abrahams 04].
- **Domain Specific Embedded Languages** are libraries meeting those criteria [Veldhuizen 95].
- **Generative Programming** is one way to design such libraries [Eisenecker 97]
- In C++, **template-based** design helps implements such libraries with high efficiency.

Generative Programming : Small Streams Make Big Rivers



Mouth.receive(Foot)

Our Strategy (~2005-2015)

- Applies DSEL generation techniques to parallel programming
- Maintains low cost of abstractions through meta-programming
- Maintains abstraction level via modern library design

Our Contributions

Tools	Publication	Scope	Applications
Quaff	ParCo'06	MPI Skeletons	Real-time 3D reconstruction
SkellBE	PACT'09	Cell BE Skeletons	Real-time Image processing
BSP++	IJPP'12	MPI/OpenMP BSP	Bioinformatics, Model Checking
NT ²	JPDC'14	Data Parallelism	Fluid Dynamics, Vision, Linear Algebra

First Disappointments

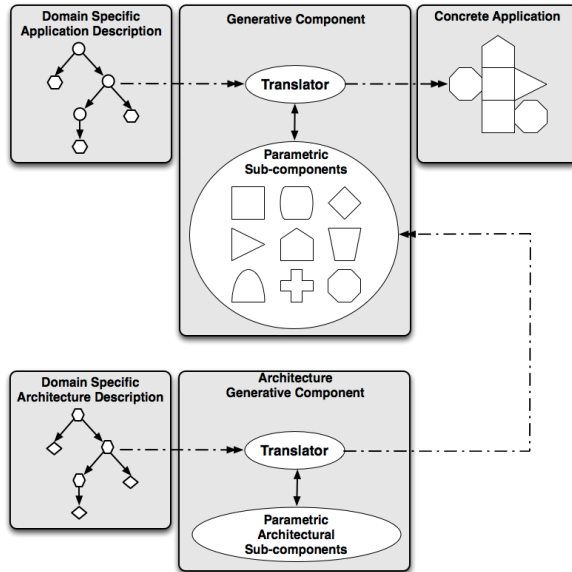
Development Limitations

- DSELs are mostly tied to the domain model.
- Architecture support is often an afterthought.
- Extensibility is difficult as many refactoring are required per architecture.
- Example : No proper way to support GPUs with those implementation techniques.

Let's Improve!

- Extends Generative Programming to take this architecture into account.
- Provides an architecture description DSEL.
- Integrates this description in the code generation process.

From DEMRAL to AA-DEMRAL



Mouth.receive(Foot2)

Post-Mortem

Tools	Issues	Change
Quaff	Skeletons at API level is impractical	Merged inside NT ²
SkellBE	Too architecture specific	Merged inside NT ²
BSP++	Integration issues	Merged inside NT ²
NT ²	Extension issues	Reuse skeletons as a base layer

Conclusion

- Skeletons are fine as parallel middleware.
- Model based abstractions are not high level enough.

Second Disappointments

The Monolith Strikes Again

- The NT²/skeleton refactoring solved a lot of issues.
- But, it turned the library into a complex monolith.
- Bug fixing and performances improvements were impractical leading to code rot.
- Feature development time increased out of proportions:
 - Support for ARM Neon - 1 months
 - Support for futures - 6 months
 - Support for CUDA - 2 years

NT² development stopped and the project was archived in late 2016

Designing HPC Tools - Attempt #2



What went wrong ?

The One Mistake

- Tool that do everything are nice on papers.
- Integration and implementation are in fact complex.
- Learning curve for new contributors is more than steep.

A New Approach (2018+)

- Software development is a complex process.
- Libraries are softwares.

What about applying some of the 30+ years old development strategies?

Building Solid Foundations

The SOLID Principles

- **Single Responsibility:** *A class should have one and only one job.*
- **Open/Close:** *Software entities should be open for extension but closed for modification.*
- **Liskov's Substitution Principle:** *Derived types should complement the base object behaviour.*
- **Interface Segregation:** *Many specific client interface are better than one general purpose interface.*
- **Dependency Inversion:** *Entities must depend on abstractions, not on concretions.*

Why SOLID?

- More a set of guidelines than a cook book.
- Prevent **design error** early on.
- Helps compare design choices.

Does those OO rules make sense to design generic libraries?

Mouth.receive(Foot3)

Our Approach

- Increased interoperability via **concepts sharing**.
- Document extension points as clearly as possible.
- Let users assemble libraries into larger abstractions.

Tools	Scope
Raberu	<i>QoL</i> : Named parameters
Kumi	<i>QoL</i> : Tuple and tuple algorithms
E.V.E	Portable SIMD abstractions and algorithms
Kiwaku (WIP)	Customisable data storage
NuCog (WIP)	Compile-time formula generator

Mouth.receive(Foot3)

E.V.E/Kumi Interoperability

```
// eve::struct_supports register a struct as a product type
struct complex : eve::struct_supports<complex,double, double="">
{
    complex(double r = 0., double i = 0.)
    {
        real(*this) = r;
        imag(*this) = i;
    }

    friend decltype(auto) real(complex&& z) { return kumi::get<0>(z); }
    friend decltype(auto) imag(complex&& z) { return kumi::get<1>(z); }
};

// EVE SIMD algorithms can consume product types stored via eve::soa_vector
eve::wide<complex>      vz = [](auto i, auto) { return complex{i+1,1./(i+1)}; }
eve::soa_vector<complex> zs = /* ... */;
eve::vector<double>      r(zs.size());
eve::transform([&](auto z) { auto[r,i] = z*vz; return r*r+i*i;}, zs, r);
```

Mouth.receive(Foot3)

Kiwaku/E.V.E/Raberu Interoperability

```
std::vector<float> d(100);

// Raberu named parameters used by KWK constructors
// KWK statically exploit raberu parameters to pick the best type to use
auto v = kwk::view{ kwk::of_size(width = 10, height = 10), source = d};

// KWK algorithms uses context as architecture-specific optimization driver
// kwk::context::simd triggers uses of EVE vectorisation process
kwk::transform( kwk::context::simd, [](auto e) { return 1.f/e*e;}, v, v);
```

From Object to Generative Programming

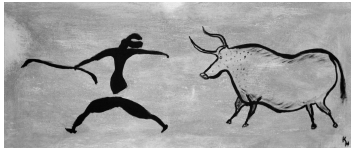
What did we learn?

- The most general SOLID rules are directly usable.
- Incorporate non-member functions into types's interfaces.
- Shift focus from Interface/Class/Object to Concept/Type/Value.
- Emphasis entities collaboration across libraries.

Our SOLID Library Principles

- **Single Responsibility:** *A **type** or **function** should have one and only one job.*
- **Open/Close:** *Software entities extension is a **core design choice**.*
- **Substitution Principle:** *Derived types should complement the base object behaviour.*
- **Interface Segregation:** *Many specific purpose **libraries** are better than a general purpose **library**.*
- **Dependency Inversion:** *Entities must depend on **concepts**, not on **types**.*

Parting Words



Maximize Interoperability!

- Languages evolved to support more generic composition mechanisms.
- **C++: We need a standard package manager!**

Nothing is too trivial!

- Don't minimize QoL tools.
- Good research means we can replicate it so don't throw code away!

Make It Known!

- The community needs more venue to **publish** and **promote** tools.
- **Github** and **JOSS** changed the landscape, we need to push through.
- Motivate your students and collaborators to communicate about their tools.

Thanks for your attention !